

Exoplanet Detection using Deep Learning

Iori Adachi Isabella Deutsch Levi Galvan Chikara Oe Zack Schuder
Linus Upson

December 15 2025

1 Introduction

The search for exoplanets has rapidly expanded over the past decade as space-based missions like *Kepler* and *TESS* have generated vast catalogs of high-precision photometry. Planetary transits, which are small periodic decreases in stellar brightness when a planet crosses the disk of its host star, remain one of the most productive detection methods for exoplanets; yet detecting these signals remains challenging. As a result, efficient and robust algorithms are needed to recover more complex transit signatures, particularly for smaller planets and long-period orbits where the signal-to-noise ratio (SNR) is low.

Historically, exoplanet transit searches have relied on matched-filter techniques such as the Box-Least-Squares (BLS) algorithm [1], which perform well for strong, box-shaped signals but struggle with low-SNR events, stellar variability, and the more realistic curved transit profiles. As our search pushes toward smaller planets and longer orbital periods, where signals are weakest, we decided to look to deep learning techniques to capture more subtle transit signatures.

Our project approaches the transit detection problem using a curated sample of Kepler stars, building consistent light curve inputs using the `lightkurve` [2] package. Because true transits are rare and small planets are especially difficult to recover, we also generate physically realistic synthetic transits using the `batman` [3] package, which models limb-darkened, non-boxlike transit shapes. Finally, we pair a CNN classifier with a Bayesian periodicity model to evaluate both signal detectability and the reliability of periodic transit claims.

Together, these approaches allow us to examine where machine-learning methods can extend the reach of traditional methods and to understand the limitations of transit recovery in two notoriously difficult categorizations.

2 Methods

2.1 Data Acquisition

We constructed an end-to-end data pipeline to retrieve, clean, and prepare Kepler light curves for machine-learning (CNN) classification and Bayesian transit recovery. The objective was to transform raw mission data into two distinct formats: fixed-length input feature vectors suitable for a CNN and raw time-series data for Bayesian inference.

Candidate stars were selected from the Kepler Object of Interest **Kepler Object of Interest (KOI)** catalog (Q1-Q8)[4]. This catalog provides essential labels, classifying targets as either "Confirmed/Candidate" (Label 1) or "False Positive" (Label 0). For each of the $\sim 3,800$ targets in our subset, we queried the Kepler archive using the `lightkurve` API. A significant challenge in Kepler data is the spacecraft's 90-degree rotation every quarter, which creates flux offsets. To address this, we downloaded all available "Quarters" for each star and stitched them together into a single, continuous light curve. To handle the large volume of data and network instability associated with the MAST archive, we implemented a robust "resume" logic. The pipeline scans the local disk for existing processed files before initiating a download, allowing the process to be stopped and restarted without data loss.

2.2 Preprocessing

Our pipeline utilizes the architecture proposed by Shallue and Vanderburg[1], splitting the input into a "Global View" and a "Local View."

Raw light curves contain low-frequency stellar variability (e.g., starspots, rotation) that can mimic or obscure transit signals. We applied a uniform cleaning process to all light curves. We utilized a standard preprocessing architecture inspired by Shallue and Vanderburg[1]:

- **Stitching:** Combining multi-quarter data into a continuous time series.
- **Detrending:** We applied a Savitzky-Golay filter (window length = 101 cadences) to flatten the light curve. A critical modification we implemented was *masking* the known transit events prior to detrending. This removes low-frequency variability while preserving the sharp, high-frequency transit dips.
- **Outlier Rejection:** A 5-sigma clipping algorithm was applied to remove cosmic ray hits and instrumental flares.
- **Normalization:** The flux was normalized to a median of 0 and scaled such that the minimum depth of the deepest transit was normalized to -1. This was essential for neural network stability.

At this stage, the pipeline splits into two paths. Unfolded (time-domain) light curves were exported immediately to CSV for the Bayesian analysis team. Simultaneously, the data was passed to the folding engine for CNN preparation.

2.3 Synthetic Transit Generation

A major limitation in exoplanet machine learning is class imbalance: confirmed planets are significantly outnumbered by non-planet stars and false positives. Furthermore, real "False Positives" often look nothing like planets, making it easy for the model to overfit. To address this, we employed the **batman** package[3] to generate physically realistic synthetic transits. We injected these simulated signals true negatives in order to convert them into true positives. This method allows the model to learn to detect planets even in the presence of realistic stellar noise.

We defined a randomized injection model with:

- **Limb Darkening:** Quadratic limb-darkening coefficients $[u_1, u_2] = [0.1, 0.3]$ to simulate the star's curvature,
- **Orbit:** Circular orbits (*eccentricity* = 0) with a fixed semi-major axis ($a/R_* = 15.0$) and fixed inclination (89.9°). Fixing these parameters standardizes the transit duration profile relative to the period.
- **Sampling:** Planet Radius (R_p/R_*) was sampled uniformly between 0.005 and 0.03 and Orbital Period sampled uniformly between 5 and 50 days.

The synthetic model was converted into a relative flux delta (dip) and **additively injected** into the detrended stellar flux. Finally, the light curve was re-normalized to ensure the global median remained at 0 and the minimum depth was scaled to -1.

2.4 Feature and Batch Construction

To prepare data for the CNN, we implemented the "Dual-View" feature construction[1]. It is a function that takes arrays of time and flux and outputs a fixed length global view (length 2001) and local view (length 201). Unlike traditional approaches that mask a specific time window, we phase-folded the data on the orbital period for both views to effectively stack all available transits, ensuring data gaps did not result in empty feature vectors.

- **Global View:** We phase-fold the full light curve on the candidate orbital period P using the provided transit epoch T_0 . This stacks all observed transits across the full time onto a single orbit. The folded light curve is then binned into a fixed number of bins (**2001 points**), to ensure consistent input shape for the CNN. This view captures the full periodic structure, allowing the model to identify stellar variability.

- **Local View:** Rather than selecting only a single transit window in the time domain, we construct the local view from the same folded light curve so that all available transits contribute. We define a symmetric window around phase 0 based on the estimated transit duration. This is a zoomed-in window centered on the transit event. The window width was defined as **4.0 times the transit duration**. This view was binned to **201 points**.

Handling False Positives (Random Folding): A critical challenge we addressed was the presence of False Positives (e.g., instrumental noise) in the catalog that lacked orbital parameters. Without a period, these could not be phase-folded. To utilize this data, we implemented a **Random Folding** strategy: for any False Positive missing a period, we assigned a random period (0.5–50 days) and epoch. When folded on these arbitrary parameters, the light curve appears as incoherent noise. This was essential to teach the CNN that "scattered, non-U-shaped noise" belongs to the negative class (Label 0).

Finally, the processed arrays were saved in compressed **.npz** batches, accompanied by a unified metadata index to allow for efficient shuffling and loading during training.

2.5 Challenges and Solutions

We initially encountered a "Class Imbalance" issue where strict filtering (e.g., removing stars without periods) resulted in a dataset containing almost exclusively confirmed planets. This caused the model to overfit, as it never learned what a "negative" sample looked like. We solved this by implementing the Random Folding strategy described above, ensuring the pipeline processed 100% of the available False Positives.

Additionally, early iterations of the "Local View" generation relied on masking a specific time window around T_0 . Due to data gaps in Kepler's downlink, this often resulted in empty arrays. We refactored the pipeline to fold the local view on the period, stacking all transits. This ensured that even if the primary transit at T_0 was missing, subsequent transits filled the gap.

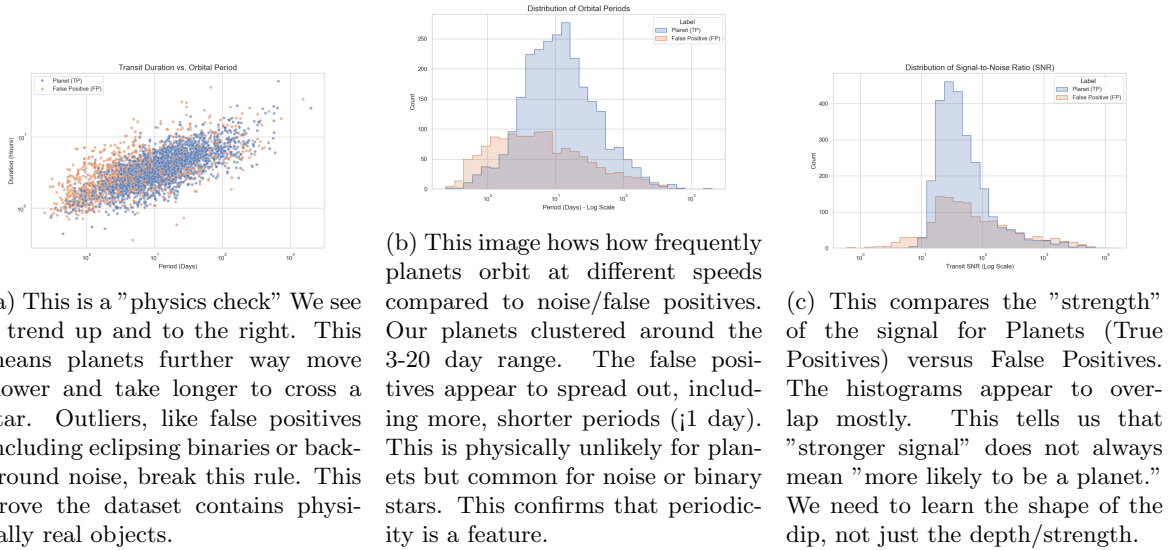


Figure 1: Insights into the Distribution and Characteristics of the Planet Candidates Versus False Positives.

3 Implementation: Bayesian Periodicity and CNN Analysis

3.1 Bayesian Periodicity Analysis

For our Bayesian analysis, we used a model for detecting general periodic signals in light curves from [5]. It is similar to the model discussed in class, assuming a stepped periodic flux curve with a number of steps, or bins, set as a model parameter. The key difference is that instead of working

in terms of a discrete signal composed of individual count detections, the model describes a signal in terms of a continuous flux with given uncertainty, which is better suited to the astrophysical examples treated in [5] and in this project. The flux in each of m bins is assumed to be normally distributed, with a mean rate r_j and a provided uncertainty. As such, the model's likelihood in terms of the periodic frequency ω , the phase shift ϕ and the bin rates is as follows

$$P(D|\omega, \phi, \vec{r}, M_m, I) = \prod_{j=1}^m \left[(2\pi)^{-n_j/2} \left(\prod_{i=1}^{n_j} \frac{1}{s_i} \right) \exp \left(-\frac{1}{2} \sum_{i=1}^{n_j} \frac{(d_i - r_j)^2}{s_i^2} \right) \right] \quad (1)$$

Where n_j is the number of observations in bin j , d_i is the measured flux of the i th observation in bin j , and s_i is its uncertainty. Assuming a uniform prior on r_j , this likelihood can be analytically marginalized over $\{r_j\}$, yielding a likelihood function for periodic binned signal irrespective of the specific bin rates.

$$P(D|\omega, \phi, M_m, I) = (2\pi)^{-N/2} (\Delta r)^{-m} \left[\prod_{i=1}^N \frac{1}{s_i} \right] \left(\frac{\pi}{2} \right)^{m/2} \exp \left(-\sum_{j=1}^m \frac{\chi_{Wj}^2}{2} \right) \times \prod_{j=1}^m \left(W_j^{-1/2} [\text{erfc}(y_{j \min}) - \text{erfc}(y_{j \max})] \right) \quad (2)$$

Where W_j , χ_{Wj}^2 , $y_{j \min}$, and $y_{j \max}$ are all quantities defined in [5] in terms of the measured fluxes in bin j and their uncertainties. Δr is the width of the prior for the r_j s.

To estimate various properties of a signal, all one needs to do is marginalize over this likelihood in certain parameters. For our purposes, the most important application is to pick out the period of transits in the light curve. This can be done by marginalizing over ϕ and M_m and then multiplying by the prior $P(\omega|I)$ in order to get a posterior probability in terms of ω .¹

$$P(\omega|D, I) = \frac{P(\omega|I)P(D|\omega, I)}{P(D|I)} \propto P(\omega|I) \sum_{m=2}^{M_{max}} \left[P(M_m|I) \int d\phi P(\phi|I) P(D|\omega, \phi, M_m, I) \right] \quad (3)$$

In this work, we assume uniform priors on ϕ and M_m and $P(\omega|I) = \frac{1}{\omega \ln \omega_{hi}/\omega_{lo}}$. The maximum value of $P(\omega|D, I)$ then yields the most likely frequency ω_{best} . To evaluate this curve numerically, we sample it at a range of ω values. Because of the typical morphology of transit events, the peak in $P(\omega|D, I)$ is often extremely narrow, so if the sampling point is even slightly off, it will miss the peak entirely. We choose our sampling density to avoid this problem. If a signal observed for total time T_{net} has true frequency ω_{true} and contains a periodic transit event with duration Δt , the first and last transit will only overlap when phase-folded if $\omega_{space} = \omega_{samp} - \omega_{true} < \omega_{true} \frac{\Delta t}{T_{net}}$. We generate samples between ω_{min} and ω_{max} with this required spacing by

$$\omega_n = \omega_{min} \left(1 + \frac{\Delta t}{T_{net}} \right)^n ; n_{max} = \text{ceil} \left(\frac{\ln(\omega_{max}/\omega_{min})}{\ln(1 + \Delta t/T)} \right) \quad (4)$$

For Δt , we choose a conservatively small estimate that hopefully balances sampling density with computational efficiency: $\Delta t = 3\text{hrs}$.

We also allow for two different models of the light curves, a non-periodic model and a constant model. Both are treated in more detail in [5], but in short, they are both special cases of the periodic model. The non-periodic model must have a period equal to the total time span under observation, and the constant model only has one bin. One can calculate the likelihoods for each of these models and compare them to the likelihood

$$P(D|M_{periodic}, I) = \int d\omega P(\omega|I) \sum_{m=2}^{M_{max}} \left[P(M_m|I) \int d\phi P(\phi|I) P(D|\omega, \phi, M_m, I) \right] \quad (5)$$

¹In practice, we discovered that marginalizing over ϕ , though theoretically justified, isn't strictly necessary for detecting periodicity, as we use a large number of bins to catch transits and each bin is treated identically. Essentially removing the integration over phi allowed for significant speed gains.

calculated by marginalizing the periodic model’s likelihood over all parameters. The odds ratios ($O_{PC} = \frac{P(D|M_{\text{periodic}}, I)}{P(D|M_{\text{constant}}, I)}$ or $O_{PNP} = \frac{P(D|M_{\text{periodic}}, I)}{P(D|M_{\text{nonperiodic}}, I)}$) generated by this comparison can, in theory, be used to determine whether the periodic model is a better fit to the data.

We also attempted to apply another variant model, this one having only two bins, a transit bin and a non-transit bin. A fraction f_{trans} of the total period is taken up by the transit bin, and the rest is contained within the non-transit bin. The calculation of the likelihood follows much the same procedure as Eqn. 2. In initial tests, it showed promise, especially for rejecting non-transit periodicity, but this approach necessitates numerically marginalizing carefully over both ϕ and f_{trans} . Combined with the fact that it doesn’t reduce the need for dense sampling in ω , the computational cost was too high to feasibly apply it to many light curves.

3.2 CNN Model Architecture

To classify transit candidates, we employ a one-dimensional convolutional neural network (CNN). The model is designed to jointly analyze both fine-scale transit morphology and long-term periodic structure by processing two representations of each light curve in parallel.

Each candidate is represented by two phase-folded inputs:

- **Local View:** a zoomed-in window centered on the transit, interpolated to 201 points, which emphasizes transit depth and overall shape.
- **Global View:** the full phase-folded light curve over the interval $[-0.5, 0.5]$, interpolated to 2001 points, which preserves long-term periodic structure, stellar variability, and potential secondary eclipses.

Both views are treated as one-dimensional time-series inputs with a single flux channel, yielding input tensors of shape (batch, 1, 201) for the local view and (batch, 1, 2001) for the global view.

The CNN consists of two independent convolutional columns that process the local and global views separately.

Local CNN Column The local column captures high-resolution transit features using a shallow convolutional stack:

- Two consecutive 1D convolutional layers ($1 \rightarrow 16$ channels, kernel size = 5), each followed by a ReLU activation.
- A max-pooling layer with kernel size 7 and stride 2 to reduce dimensionality while preserving transit morphology.

This produces an output tensor of shape (batch, 16, 94), which is then flattened into a 1,504-dimensional feature vector.

Global CNN Column The global column captures long-range patterns using a deeper architecture:

- Two 1D convolutional layers ($1 \rightarrow 16$ channels, kernel size = 5) with ReLU activations, followed by max pooling (kernel size = 5, stride = 2).
- A second convolutional block consisting of two layers ($16 \rightarrow 32$ channels, kernel size = 5), again followed by ReLU activations and max pooling (kernel size = 5, stride = 2).

The resulting tensor has shape (batch, 32, 492) and is flattened into a 15,744-dimensional feature vector.

Finally, the flattened feature vectors from the local and global columns are concatenated to form a combined 17,248-dimensional representation. This fused feature vector is passed through a fully connected multilayer perceptron (MLP) head consisting of:

- Three hidden layers, each with 512 neurons and ReLU activation,
- A final linear layer producing a single scalar output.

The final layer outputs a logit. And we transform it to the probability that the input light curve contains a planetary transit by sigmoid. Also the predicted label from model was calculated by equation 6.

$$\text{predicted label} = \begin{cases} 1 & \text{if } p > 0.5, \\ 0 & \text{if } p \leq 0.5. \end{cases} \quad (6)$$

In order to calculate loss, no activation function is applied at the final output layer; instead, binary classification is performed using a `BCEWithLogitsLoss`, which internally applies a sigmoid transformation during training and is more numerically stable than normal `BCELoss`.

4 Evaluation of Results

4.1 Bayesian Model

For each normalized light curve, we calculated $P(\omega|D, I)$ and the odds ratios O_{PC} and O_{PNP} using the prescriptions in 3.1, with $\omega_{min} = \frac{2\pi}{100\text{days}}$ and $\omega_{max} = \frac{2\pi}{2\text{days}}$. We ran into significant computational challenges, as a typical light curve had ~ 40000 flux measurements and required ~ 30000 ω samples to resolve. These were partially alleviated by parallel evaluation of the likelihood and JIT compilation of time-intensive functions, but the time requirements were still high.

We found that the Bayesian pipeline struggled on many of the lightcurves in our dataset, because of various violations of the model’s assumptions. The model often picks out other sources of periodicity with very different characteristics from transiting exoplanets. In addition, many of the light curves in our dataset had significant non-periodic variation. Neither of these possibilities are accounted for in the relatively simple Bayesian models we used. Both of these deviations from our model assumptions meant that both O_{PC} or O_{PNP} were poor metrics for detecting exoplanets. For many light curves, one or both odds ratios are high, signifying a positive detection, but looking at the light curve folded by the best-fit frequency, it’s clear that the pipeline detected some other periodic feature, such as a roughly sinusoidal variation in flux.

The more useful indicator of whether the model picked out a transit was the appearance of a characteristic short drop in the intensity when folded by the best-fit period. Of the light curves that weren’t dominated by another non-transit periodicity or non-periodic features, the model correctly identified the period of 69% of the exoplanets. Almost all of the ones that it didn’t correctly identify were in the low-SNR regime. Typical examples of positive and incorrect detections are shown below in Figs. 2 and 3. Using the existence of this single dip in the folded signal as a metric, the model achieves near-100% precision, if not recall.

We believe that many of the times that the model missed the true periodicity of a star, it was due to insufficiently dense sampling in ω . When the SNR is low, our intuition is that it should make the peak in the posterior even thinner than it already is. We were constrained by the computational time needed to run denser sampling, but we believe we could have made significant accuracy gains if we had access to more CPU cores.

Bayesian Periodicity Analysis for KIC 3865567

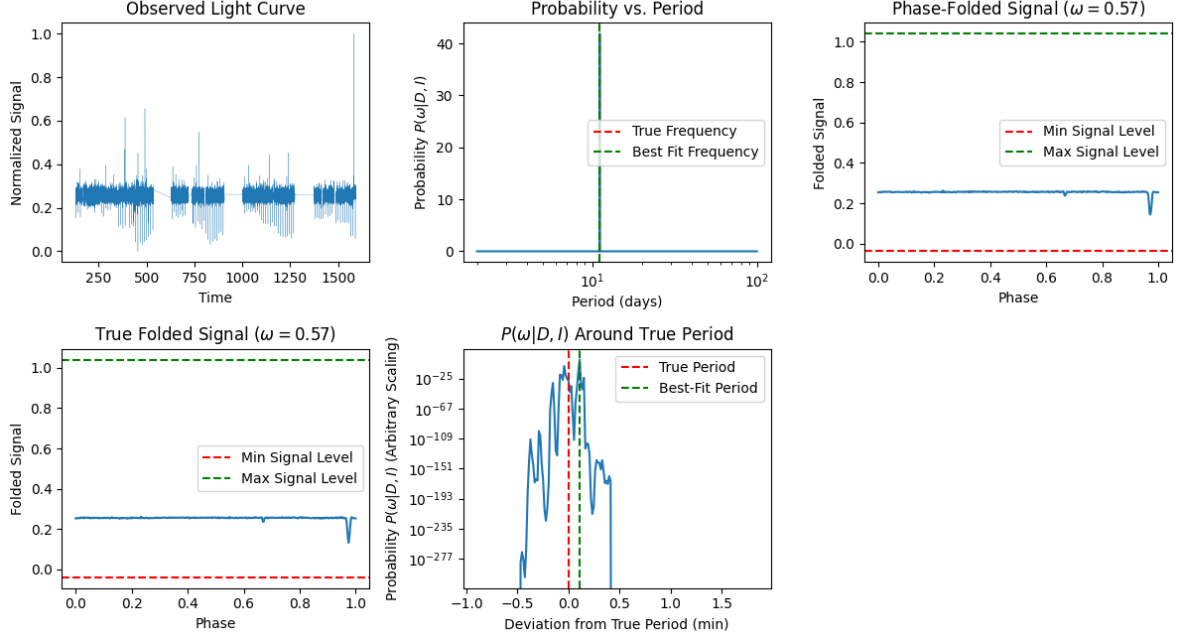


Figure 2: Example of successful Bayesian periodicity detection on KIC 3865567. $P(\omega|D, I)$ plotted against period for readability. Top right shows the signal folded by the best-fit period while bottom left shows it folded by the true period. Note in top center, bottom center how narrow the peak in the posterior probability is.

Bayesian Periodicity Analysis for KIC 10600955

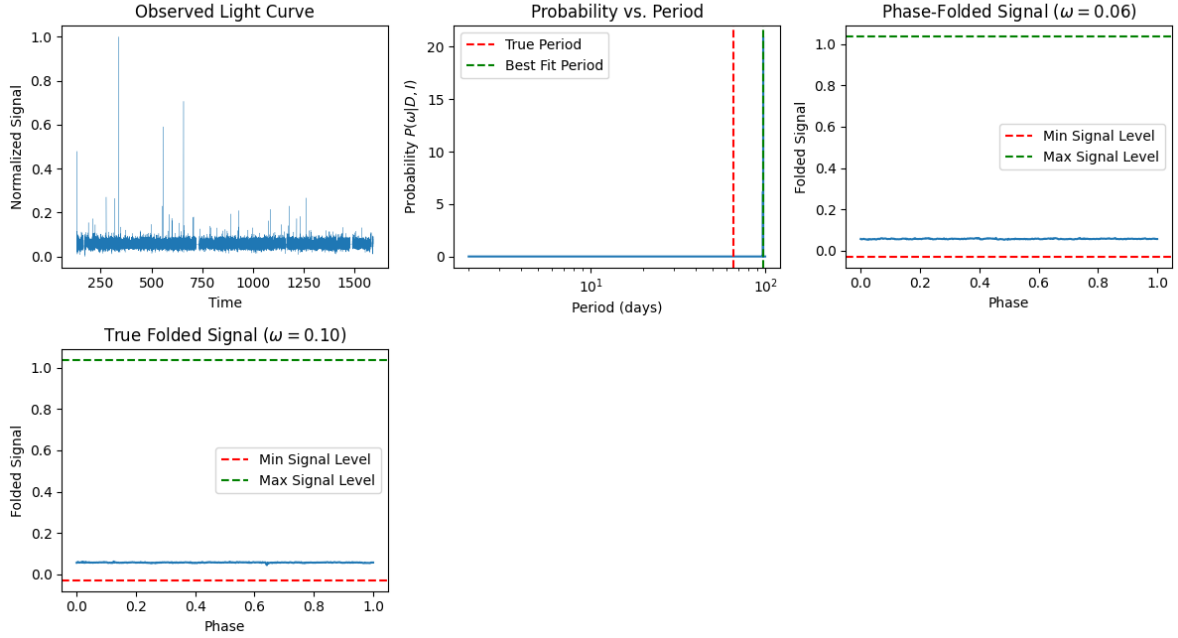


Figure 3: Example of unsuccessful periodicity detection on KIC 10600955. Note the high noise and the low depth of the transit, which is barely visible in the signal folded by the true period.

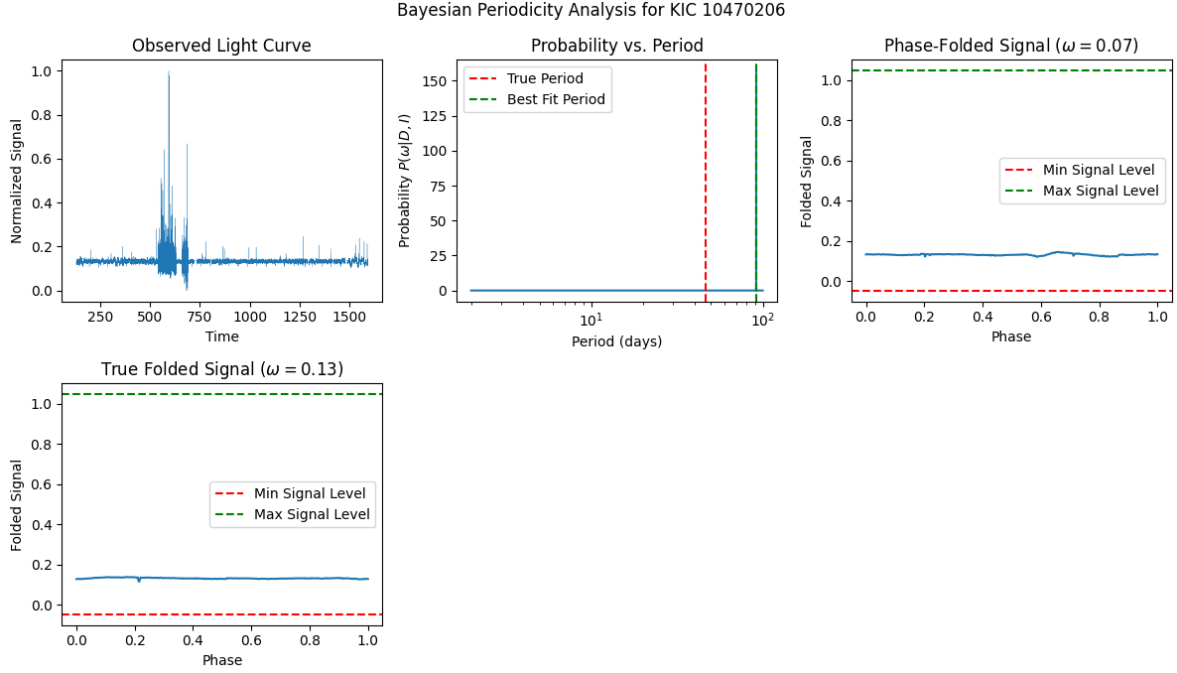


Figure 4: In this example, the Bayesian pipeline converged on a period that was exactly twice the true value. Note the high SNR in the light curve and the two transits (separated by half a period) when the signal is folded by the best fit phase. This is expected behavior, and typically easily noticeable.

4.2 CNN

For the CNN, we used the following hyper parameter values.

- LEARNING RATE = 0.001
- NUM EPOCHS = 50
- BATCH SIZE = 64

The dataset includes 182 samples of label 0(False Positives) and 983 samples of label 1(Positives). We trained the CNN using the above parameters and architectures described in Section 3.2. Figure 5 shows the results, where the accuracy went up and loss went down during training. However, validation accuracy did not show clear improvement and was fluctuating around 0.85, as well as the validation loss which increased. Each Recall is calculated by equation 7.

$$Recall_1 = TP/(TP + FN), \quad Recall_0 = TN/(TN + FP) \quad (7)$$

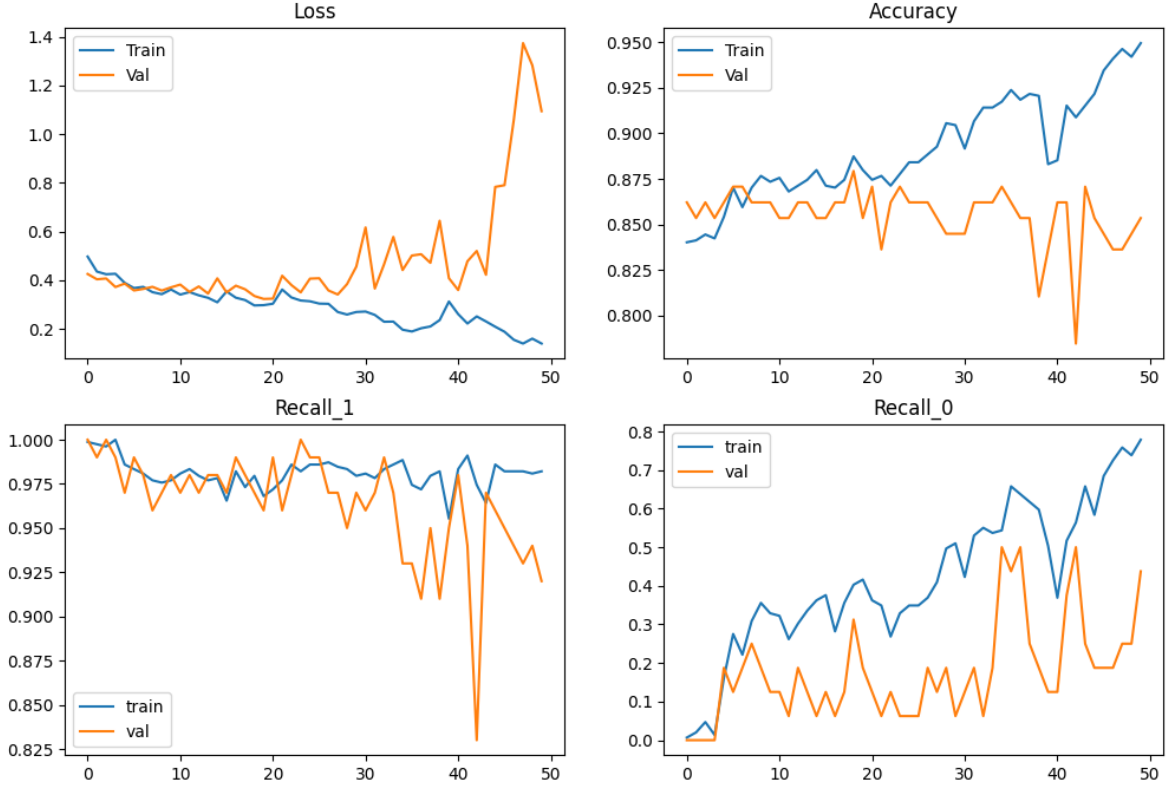


Figure 5: Training and validation performance of the CNN over 50 epochs. Shown are the loss, accuracy, recall for the positive (planet) class, and recall for the false positive class.

The test results are as follows: $accuracy = 0.8205$, $Precision_1 = 0.8990$, $Recall_1 = 0.8900$, $Recall_0 = 0.4118$, $F1 = 0.8945$. $Precision$ and $F1$ is calculated by equation 8.

$$Precision_1 = TP / (TP + FP), \quad F1 = 2 \cdot Precision_1 \cdot Recall_1 / (Precision_1 + Recall_1) \quad (8)$$

Also, the confusion matrix for the test data is shown in Figure 6.

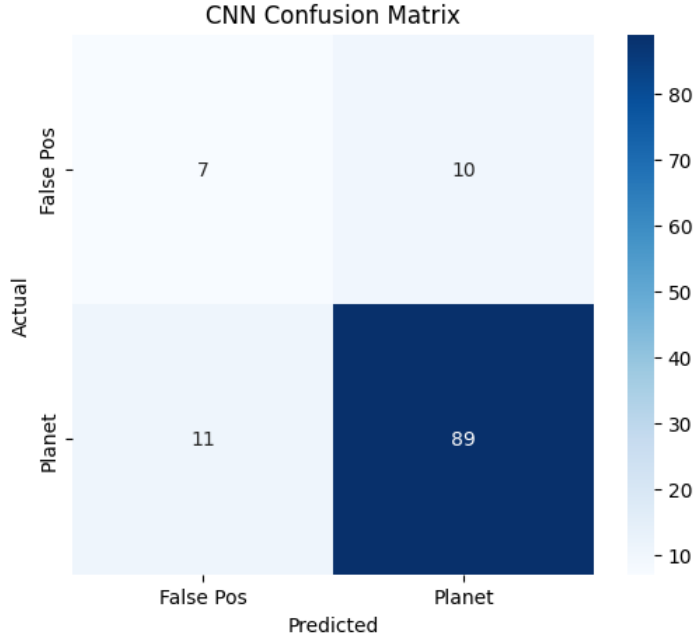


Figure 6: Confusion matrix for the CNN evaluated on the test dataset.

In the Appendix 6, we present the feature maps, which are the output of local and global convolutional networks(right before the MLP). Each filter captures different features.

5 Final Discussion

The Bayesian analysis provided a crucial physical validation of our pipeline. It successfully recovered orbital periods for high signal-to-noise ratio candidates confirms that our de-trending and stitching processes preserved the underlying physical signal. Despite all that, there was significant room for improvement. If the signal was dominated by non-periodic variation or other periodic variations unconnected with exoplanet transits, the Bayesian pipeline struggled. In addition, applying Bayesian methods to this problem is computationally expensive. We believe the time needed to run intensive inference was a significant constraint on our accuracy.

The CNN results can be explained as follows. First, the proportion of label 1 in the training and validation datasets are approximately 0.84 and 0.86, respectively, indicating that majority of the original data belong to label 1. At first, both training and validation accuracies appear very high. However, the initial accuracy is essentially equal to the proportion of label 1 in the dataset. Our model can achieve high accuracy simply by predicting 1. Next, please look at the chart of Recall in Figure 5. Validation $Recall_1$ decreased and $Recall_0$ increased. This means that the model gradually started missing class 1, while making fewer mistakes on class 0. We can also understand it from the loss curve. As the model also started predicting label 0, the predicted probabilities moved close to 0.5, which led to an increase in loss when the predictions were incorrect. In terms of the validation accuracy curve, the initial accuracy might be just too high and the model couldn't get higher accuracy than it. From the test results, the model misses false positives more frequently than true positives which is also due to the class imbalance in the dataset.

But, the increase of recall 0 suggests that the "Dual-View" architecture (Global + Local) is somewhat effective at distinguishing planetary transit shapes from stellar noise. The high training accuracy (95%) compared to the validation accuracy suggests some overfitting, which is expected given the limited dataset size compared to the full Kepler catalog. Permitted more time, we would let our current model process the data for longer or we would find better strategies for acquiring enough data and cleaning it. We would expand the synthetic injection pipeline to generate a larger, more diverse training set rather than relying solely on the limited number of Kepler planets.

We demonstrated that these two methods work best together. The CNN can be a fast preliminary

assessment system (processing thousands of stars in minutes), while Bayesian methods can provide more precise orbital parameter estimation, including a rigorous posterior on the period (See bottom center panel of Fig. 2).

A large portion of our positive class (Planets) was generated using the `batman` package. While physically accurate and necessary at times, these injections are "cleaner" than real Kepler data. It introduced a potential bias. Real planets often have transit time variations or stellar activity distortions that our synthetic set lacks. This could indicate our model's performance on other data might be slightly lower than the test metrics suggest.

To improve the model in future iterations, we would explicitly include Eclipsing Binaries in the negative class. Currently, our "Random Folding" strategy teaches the model to reject incoherent noise. This would force the CNN to learn the subtle depth and shape differences. In this study, we focus on CNN; however, exploring ResNet, LSTM, and their combination is left for future work. Additionally, we could also adapt the `lightcurve` download function to fetch TESS data, testing if a model trained on Kepler's long cadence data can generalize to TESS's short cadence targets.

References

- [1] C.J. Shallue and A. Vanderburg, *Identifying Exoplanets with Deep Learning: A Five-Planet Resonant Chain around K2-138*, *The Astronomical Journal*, **155**, 94 (2018). Available at: <https://ui.adsabs.harvard.edu/abs/2018AJ....155...94S/abstract>.
- [2] Lightcurve Collaboration, *lightcurve: Kepler and TESS Time Series Analysis in Python* (2018). GitHub Repository. Available at: <https://github.com/lightcurve/lightcurve>.
- [3] Kreidberg, L., *batman: Basic Transit Model cAlculationN in Python* (2015). Published in the Publications of the Astronomical Society of the Pacific. Available at: <https://ui.adsabs.harvard.edu/abs/2015PASP...127.1161K/abstract>.
- [4] VijayVeersingh, *Kepler and TESS Exoplanet Data* (2020). Kaggle Dataset. Available at: <https://www.kaggle.com/datasets/vijayveersingh/kepler-and-tess-exoplanet-data>.
- [5] Gregory, P.C., *Bayesian Periodic Signal Detection. I. Analysis of 20 Years of Radio Flux Measurements of the X-Ray Binary LS I +61°303* (1999). Published in The Astrophysical Journal. Available at: <https://iopscience.iop.org/article/10.1086/307433>.
- [6] SauravSJK, *Exoplanet-Habitability-Prediction* (2021). GitHub Repository. Available at: <https://github.com/SauravSJK/Exoplanet-Habitability-Prediction>.
- [7] S. Basak, S. Agrawal, S. Saha, A.J. Theophilus, K. Bora, G. Deshpande, and J. Murthy, *Habitability Classification of Exoplanets: A Machine Learning Insight*, arXiv:1805.08810 (2018). Available at: <https://arxiv.org/abs/1805.08810>.
- [8] R. Misra, *Predicting Habitable Exoplanets from NASA's Kepler Mission Data Using Machine Learning*, CS229 Final Project Report, Stanford University (2017). Available at: <https://cs229.stanford.edu/proj2017/final-reports/5228128.pdf>.

6 Appendix

The feature map of local convolutional network are shown in Figure 7 and Figure 8. Each number in figure shows the different output from different filter.(output of local convolutional network has its shape (16, 94))

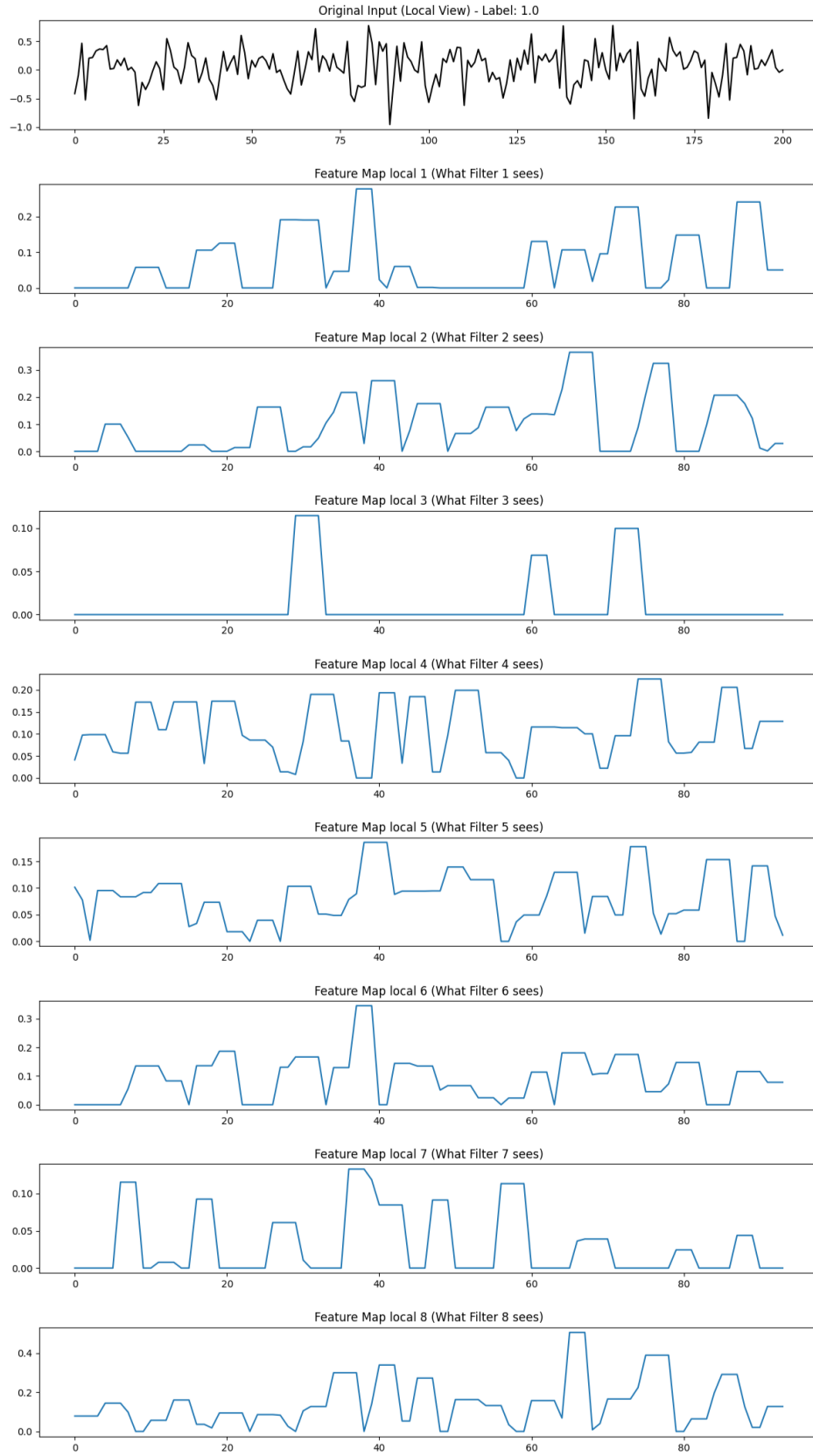


Figure 7: Local feature maps for each channel(original input and channel 1–8)

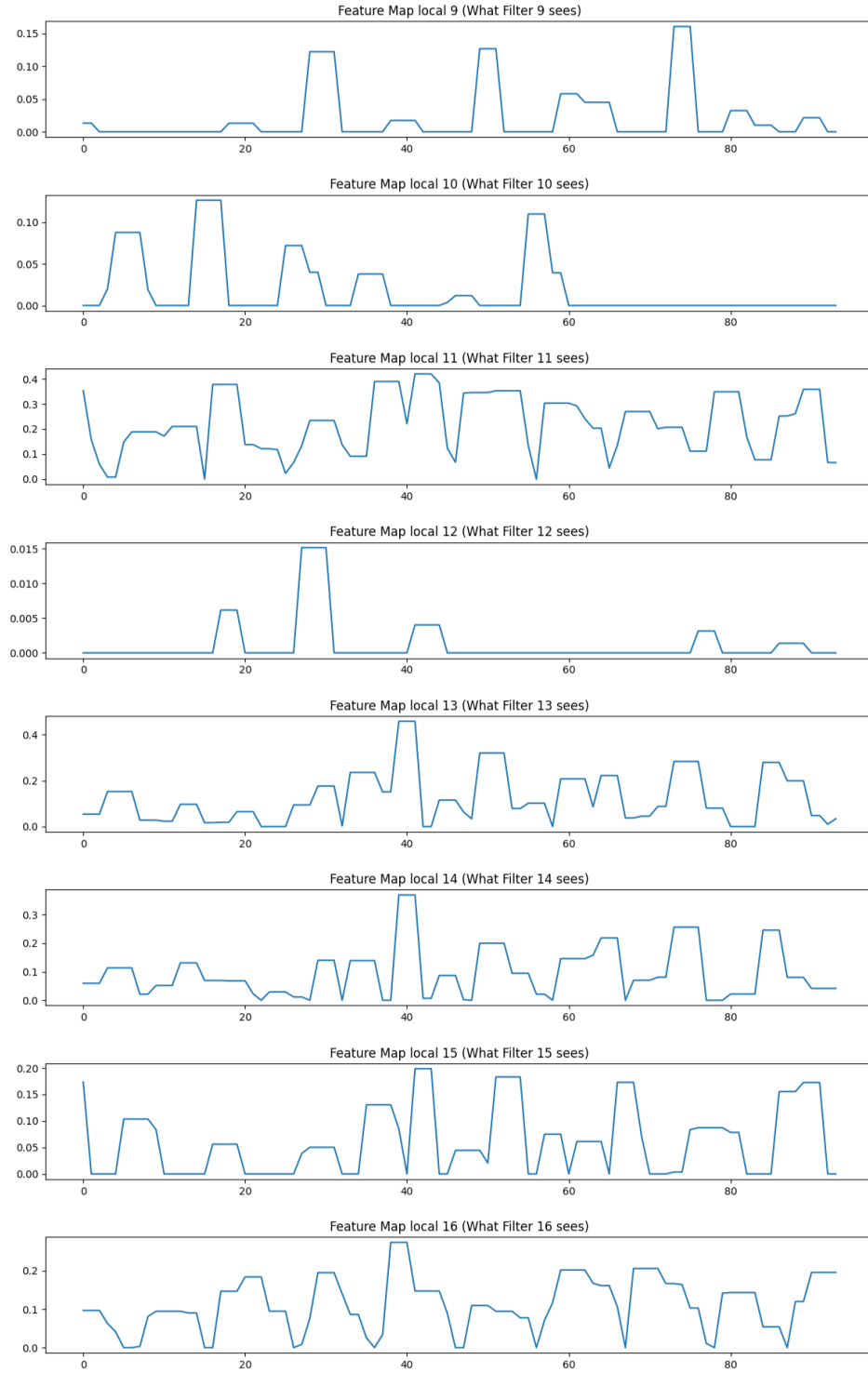


Figure 8: Local feature maps for each channel(channel 9–16)

The feature map of global convolutional network are shown in Figure 9, Figure 10, Figure 11, and Figure 12. Again, each number in figure shows the different output from different filter.(output of local convolutional network has its shape (32, 492))

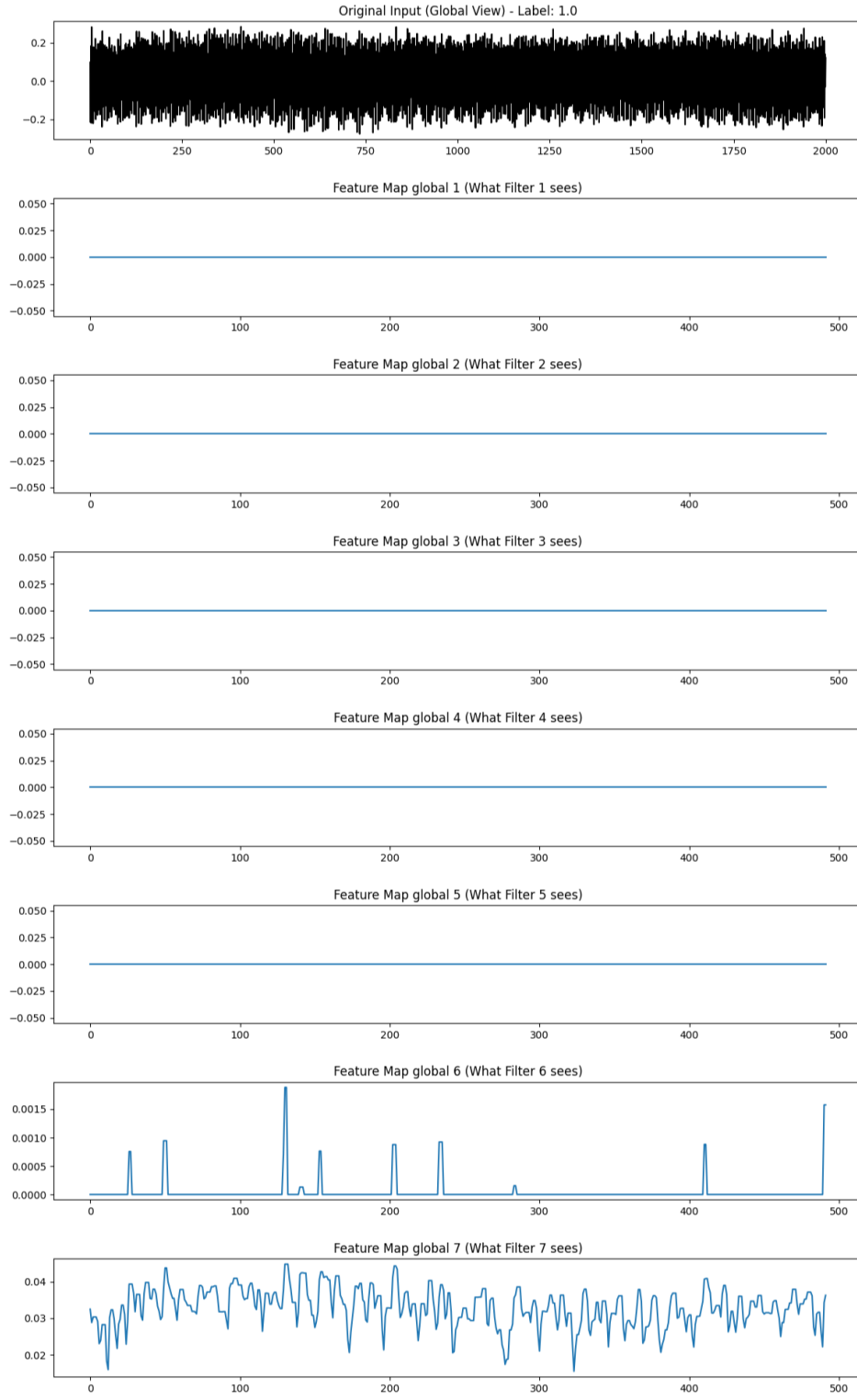


Figure 9: Global feature maps for each channel(original input and channel 1–7)

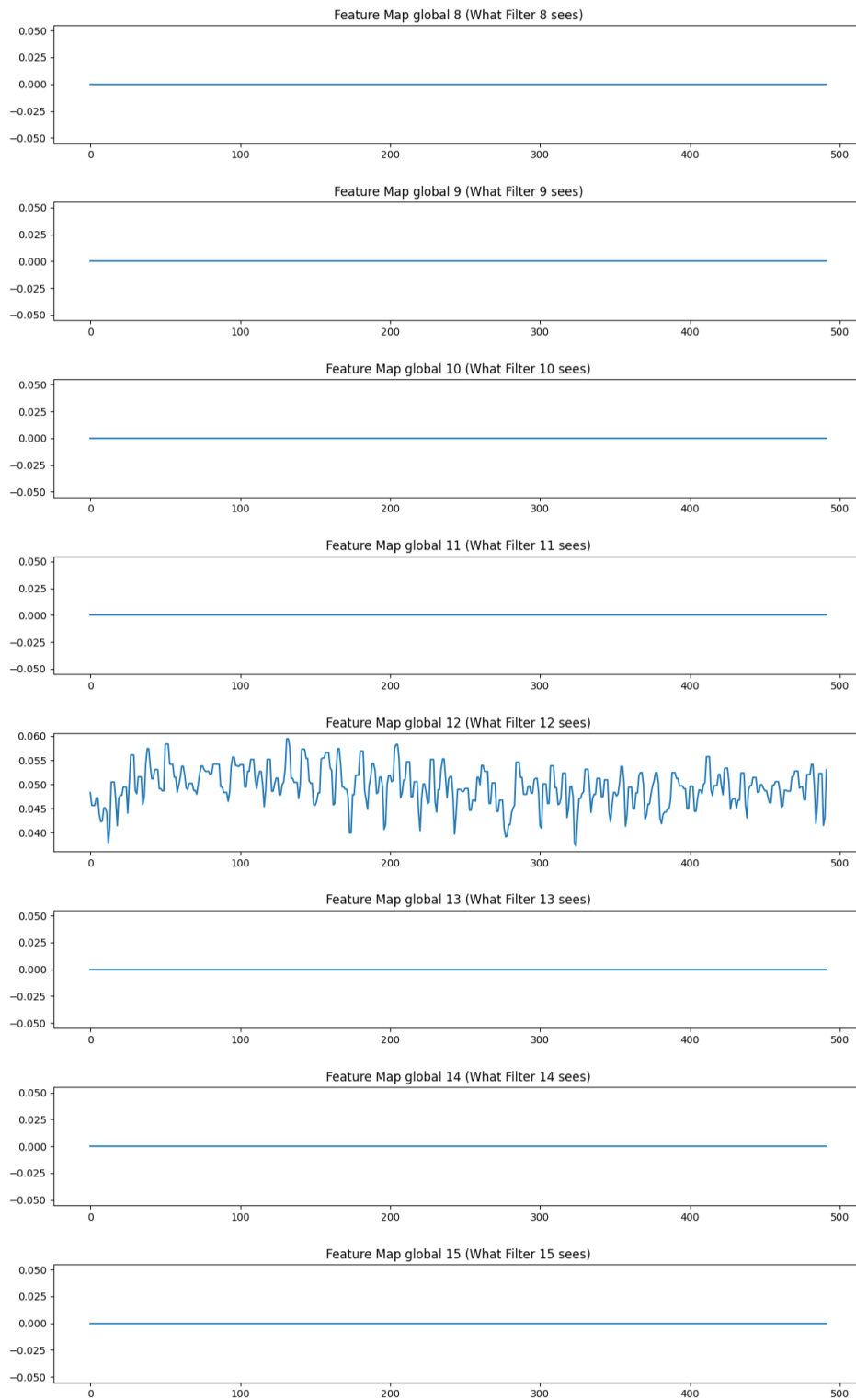


Figure 10: Global feature maps for each channel(channel 8-15)

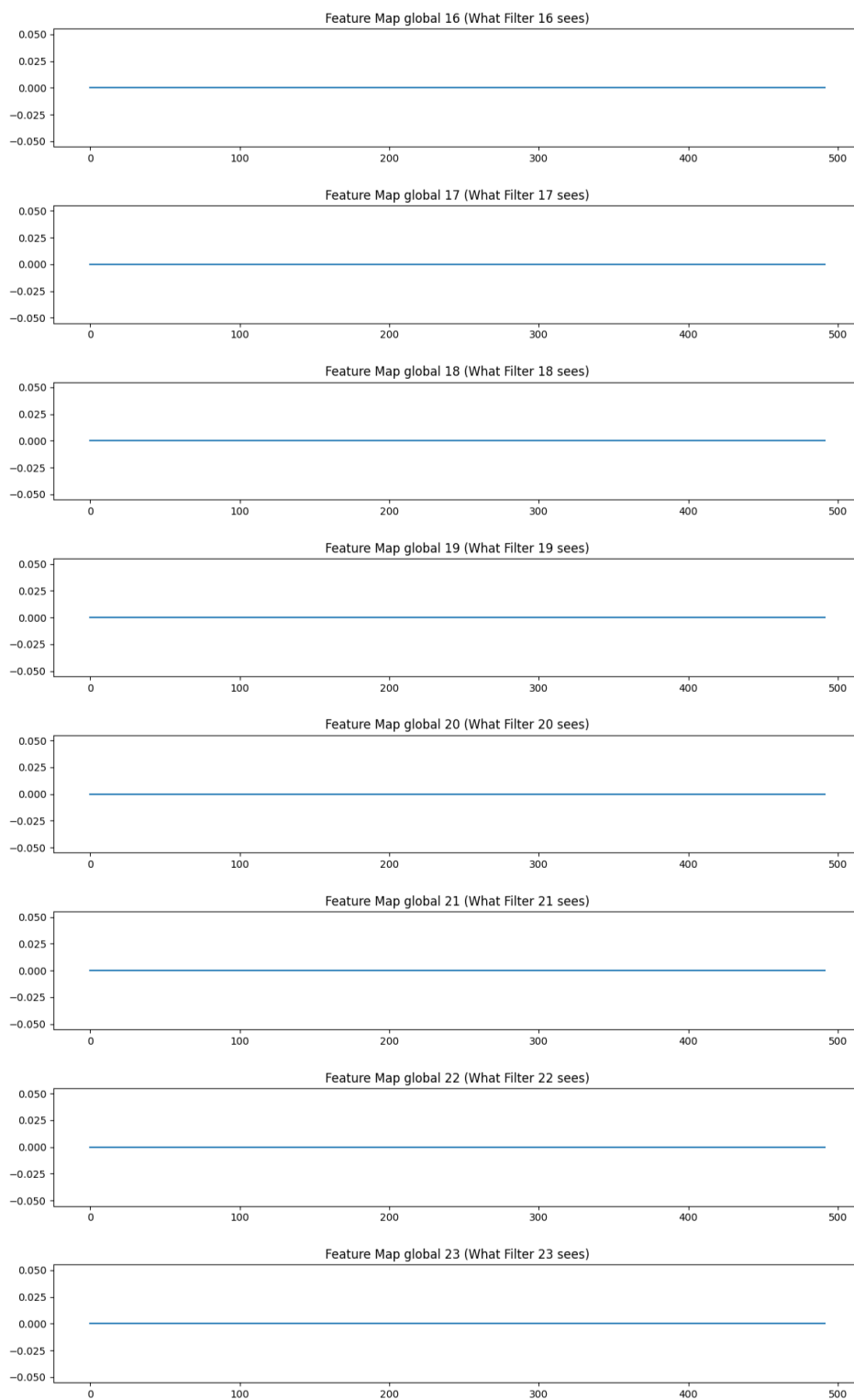


Figure 11: Global feature maps for each channel(channel 16–23)

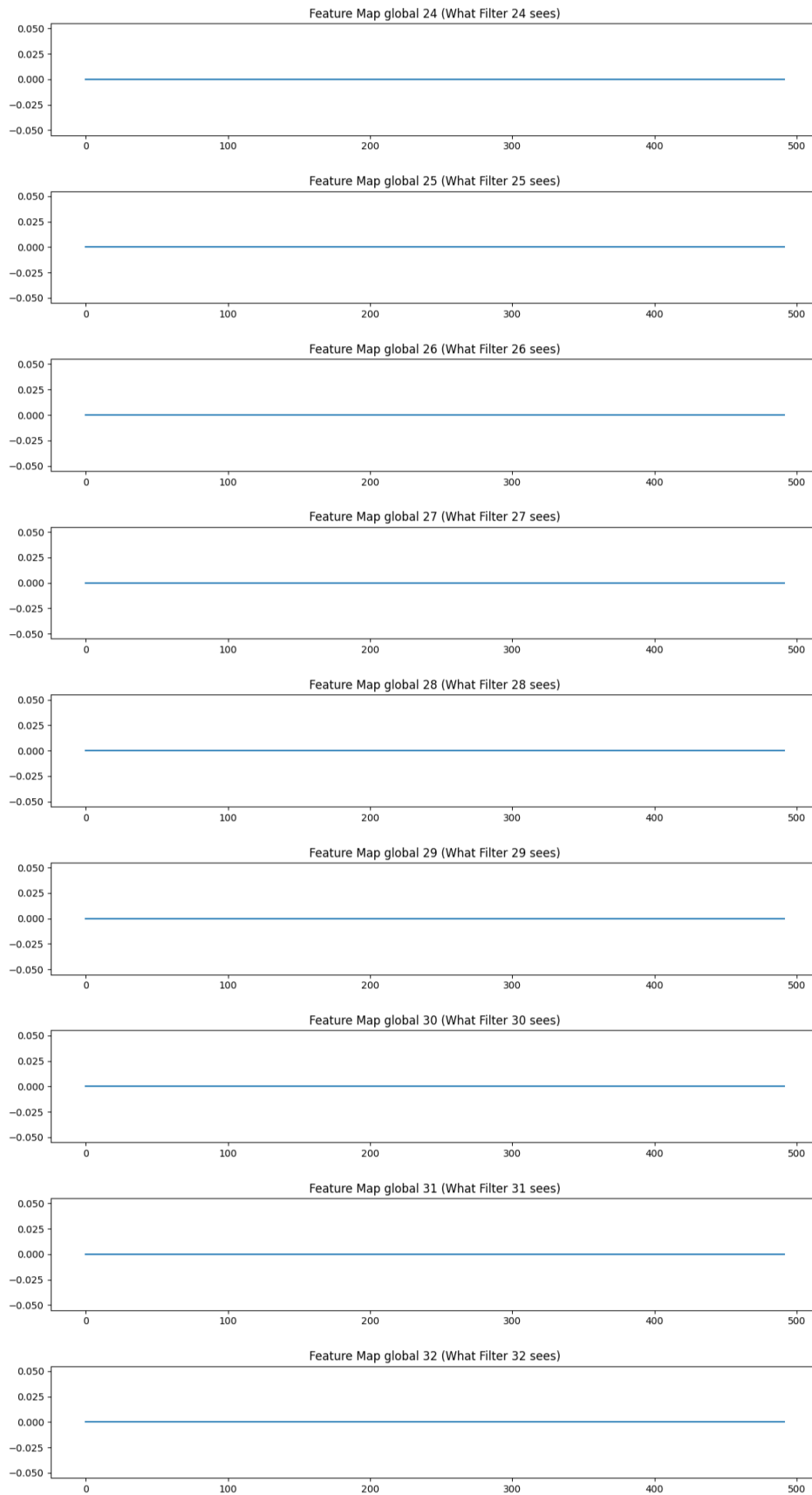


Figure 12: Global feature maps for each channel(channel 24–32)